

Preparation

```
# Dependencies

# Standard Dependencies
import os
import numpy as np
import pandas as pd
from math import sqrt

# Visualization
from pylab import *
import matplotlib.mlab as mlab
import matplotlib.pyplot as plt
import seaborn as sns

!pip install scipy

Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (1.16.3)
Requirement already satisfied: numpy<2.6,>=1.25.2 in
/usr/local/lib/python3.12/dist-packages (from scipy) (2.0.2)

# Statistics
from statistics import median
from scipy import signal
from scipy.special import factorial
import scipy.stats as stats
from scipy.stats import sem, binom, lognorm, poisson, bernoulli,
spearmanr
from scipy.fftpack import fft, fftshift
# Scikit-learn for Machine Learning model

# Read in csv of Toy Dataset
# We will use this dataset throughout the tutorial
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
```

PMF (Probability Mass Function)

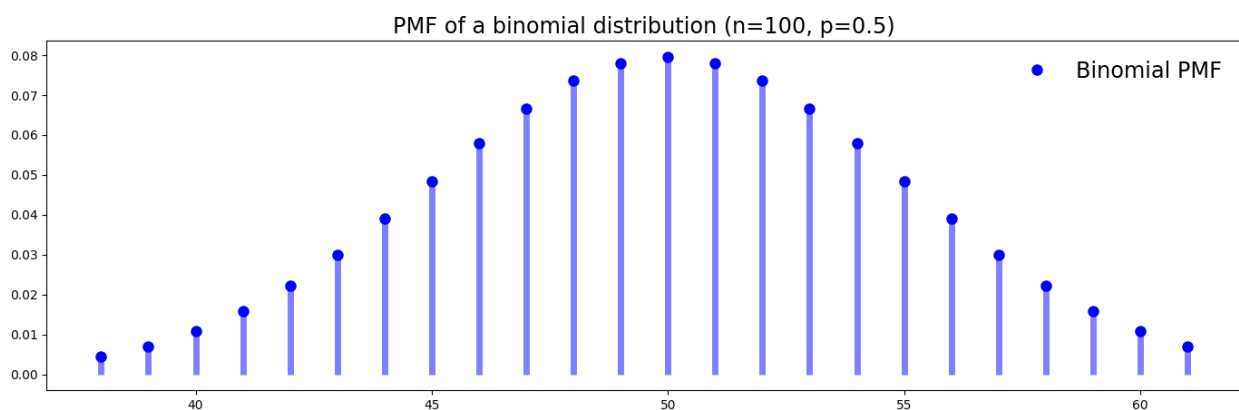
Here we visualize a PMF of a binomial distribution. You can see that the possible values are all integers. For example, no values are between 50 and 51.

The PMF of a binomial distribution in function form:

See the "[Distributions](#)" section for more information on binomial distributions.

```
# PMF Visualization
n = 100
p = 0.5

fig, ax = plt.subplots(1, 1, figsize=(17,5))
x = np.arange(binom.ppf(0.01, n, p), binom.ppf(0.99, n, p))
ax.plot(x, binom.pmf(x, n, p), 'bo', ms=8, label='Binomial PMF')
ax.vlines(x, 0, binom.pmf(x, n, p), colors='b', lw=5, alpha=0.5)
rv = binom(n, p)
#ax.vlines(x, 0, rv.pmf(x), colors='k', linestyle='--', lw=1,
#label='frozen PMF')
ax.legend(loc='best', frameon=False, fontsize='xx-large')
plt.title('PMF of a binomial distribution (n=100, p=0.5)',
          fontsize='xx-large')
plt.show()
```



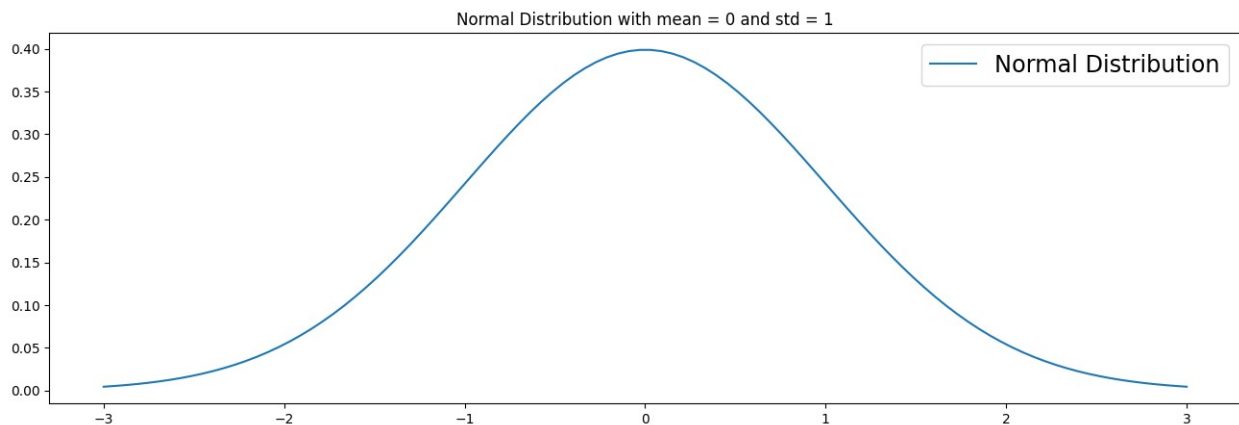
PDF (Probability Density Functions)

The PDF is the same as a PMF, but continuous. It can be said that the distribution has an infinite number of possible values. Here we visualize a standard normal distribution with a mean of 0 and standard deviation of 1.

PDF of a normal distribution in formula form:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(\mu - x)^2}{2\sigma^2}}$$

```
# Plot normal distribution
mu = 0
variance = 1
sigma = sqrt(variance)
x = np.linspace(mu - 3*sigma, mu + 3*sigma, 100)
plt.figure(figsize=(16,5))
plt.plot(x, stats.norm.pdf(x, mu, sigma), label='Normal Distribution')
plt.title('Normal Distribution with mean = 0 and std = 1')
plt.legend(fontsize='xx-large')
plt.show()
```



CDF (Cumulative Distribution Function)

The CDF maps the probability that a random variable X will take a value of less than or equal to a value x ($P(X \leq x)$). CDF's can be discrete or continuous. In this section we visualize the continuous case. You can see in the plot that the CDF accumulates all probabilities and is therefore bounded between $0 \leq x \leq 1$.

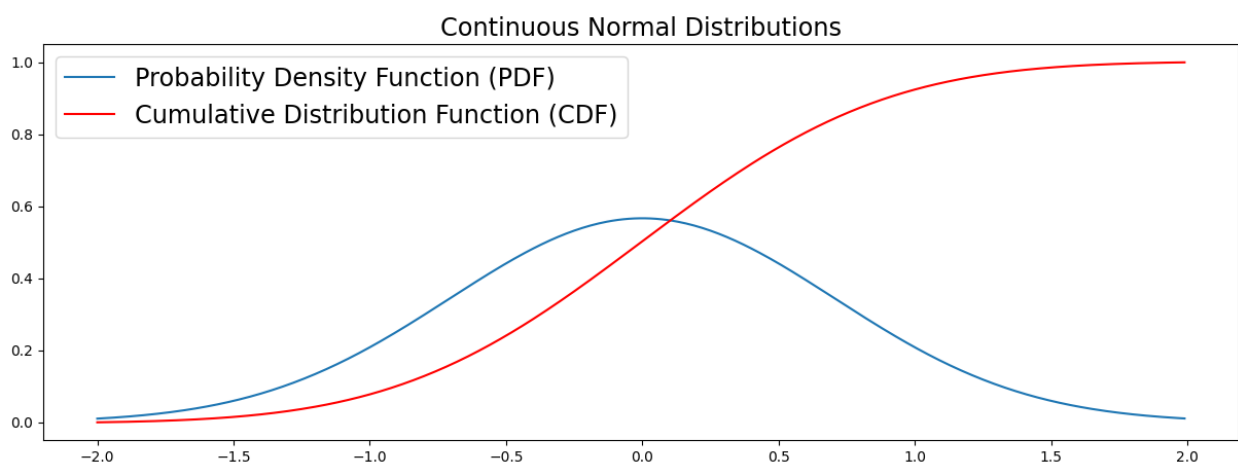
The CDF of a normal distribution as a formula:

Note: erf means "*error function*".

```
# Data
X = np.arange(-2, 2, 0.01)
Y = exp(-X ** 2)

# Normalize data
Y = Y / (0.01 * Y).sum()

# Plot the PDF and CDF
plt.figure(figsize=(15,5))
plt.title('Continuous Normal Distributions', fontsize='xx-large')
plot(X, Y, label='Probability Density Function (PDF)')
plot(X, np.cumsum(Y * 0.01), 'r', label='Cumulative Distribution
Function (CDF)')
plt.legend(fontsize='xx-large')
plt.show()
```



Distributions

A Uniform distribution is pretty straightforward. Every value has an equal chance of occurring. Therefore, the distribution consists of random values with no patterns in them. In this example we generate random floating numbers between 0 and 1.

The PDF of a Uniform Distribution:

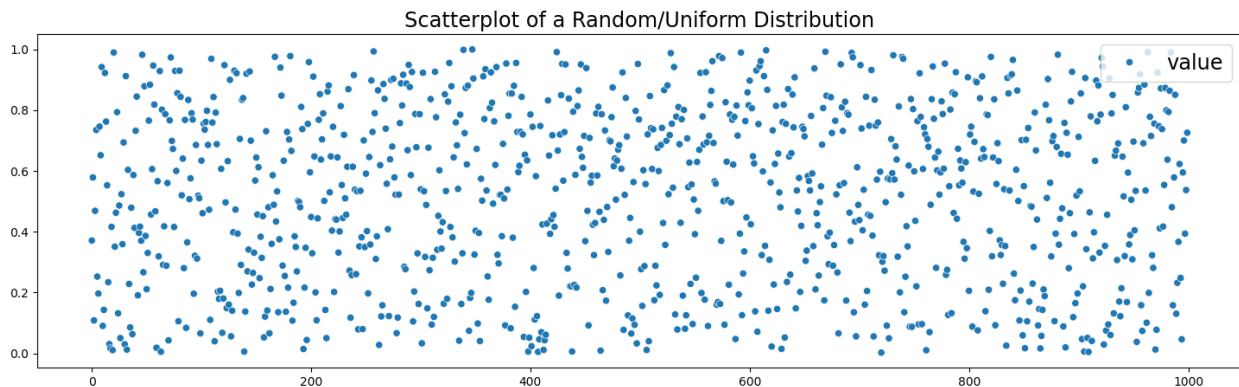
CDF:

```
# Uniform distribution (between 0 and 1)
uniform_dist = np.random.random(1000)
uniform_df = pd.DataFrame({'value' : uniform_dist})
uniform_dist = pd.Series(uniform_dist)

plt.figure(figsize=(18,5))
sns.scatterplot(data=uniform_df)
```

```
plt.legend(fontsize='xx-large')
plt.title('Scatterplot of a Random/Uniform Distribution',
          fontsize='xx-large')
```

```
Text(0.5, 1.0, 'Scatterplot of a Random/Uniform Distribution')
```



```
plt.figure(figsize=(18,5))
sns.distplot(uniform_df)
plt.title('Random/Uniform distribution', fontsize='xx-large')
```

```
/tmp/ipykernel_9909/1098536719.py:2: UserWarning:
```

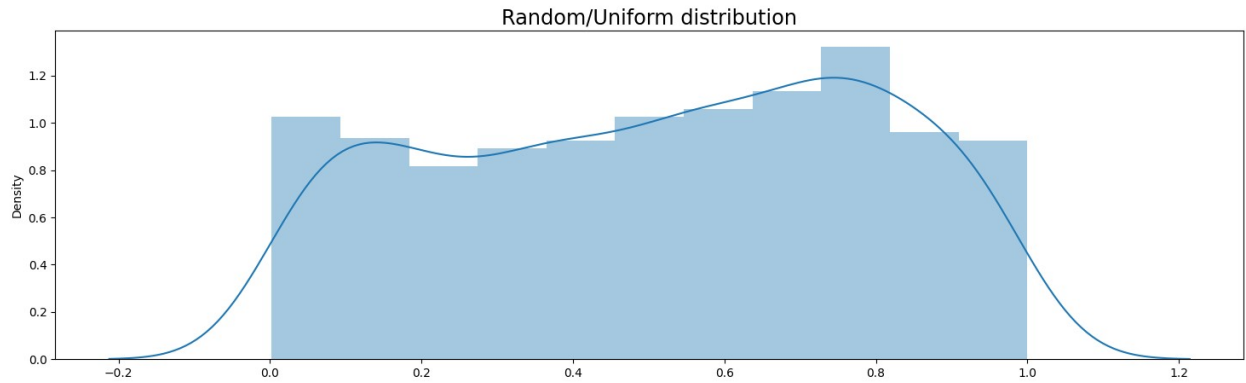
```
`distplot` is a deprecated function and will be removed in seaborn
v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(uniform_df)
```

```
Text(0.5, 1.0, 'Random/Uniform distribution')
```



Normal Distribution

A normal distribution (also called Gaussian or Bell Curve) is very common and convenient. This is mainly because of the [Central Limit Theorem \(CLT\)](#), which states that with a large amount of independent random variables (like coin flips) the distribution tends towards a normal distribution.

PDF of a normal distribution:

CDF:

```

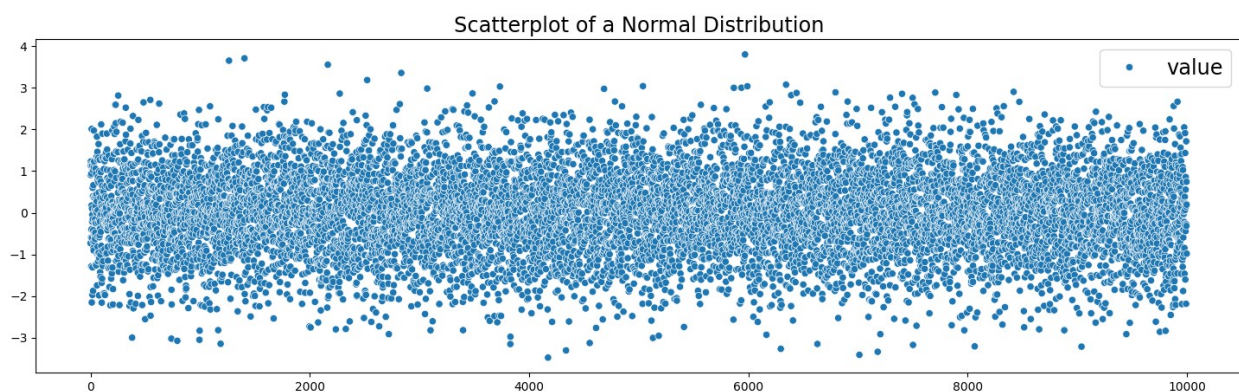
# Generate Normal Distribution
normal_dist = np.random.randn(10000)
normal_df = pd.DataFrame({'value' : normal_dist})
# Create a Pandas Series for easy sample function
normal_dist = pd.Series(normal_dist)

normal_dist2 = np.random.randn(10000)
normal_df2 = pd.DataFrame({'value' : normal_dist2})
# Create a Pandas Series for easy sample function
normal_dist2 = pd.Series(normal_dist2)

normal_df_total = pd.DataFrame({'value1' : normal_dist,
                                'value2' : normal_dist2})

# Scatterplot
plt.figure(figsize=(18,5))
sns.scatterplot(data=normal_df)
plt.legend(fontsize='xx-large')
plt.title('Scatterplot of a Normal Distribution', fontsize='xx-large')
Text(0.5, 1.0, 'Scatterplot of a Normal Distribution')

```



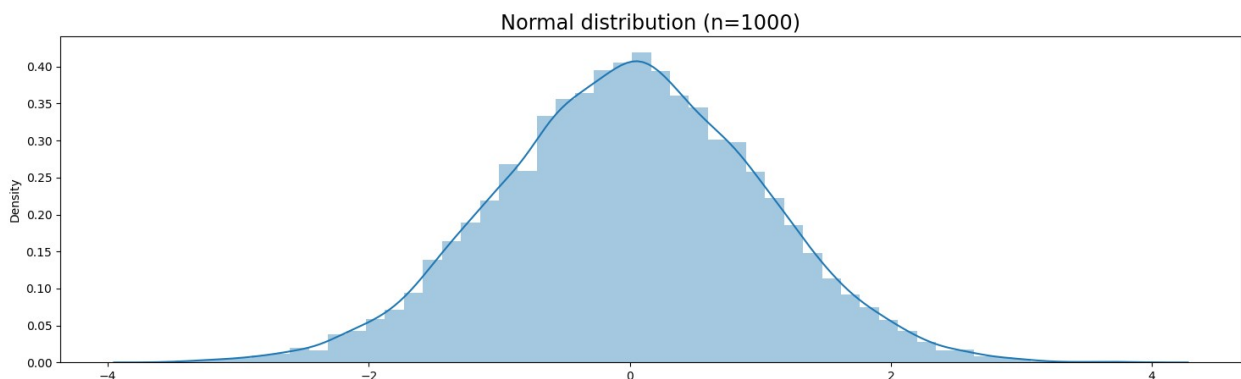
```
# Normal Distribution as a Bell Curve
plt.figure(figsize=(18,5))
sns.distplot(normal_df)
plt.title('Normal distribution (n=1000)', fontsize='xx-large')

/tmp/ipykernel_9909/2492441267.py:3: UserWarning:
`distplot` is a deprecated function and will be removed in seaborn
v0.14.0.
```

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(normal_df)
Text(0.5, 1.0, 'Normal distribution (n=1000)')
```



Binomial Distribution

A Binomial Distribution has a countable number of outcomes and is therefore discrete.

Binomial distributions must meet the following three criteria:

1. The number of observations or trials is fixed. In other words, you can only figure out the probability of something happening if you do it a certain number of times.
2. Each observation or trial is independent. In other words, none of your trials have an effect on the probability of the next trial.
3. The probability of success is exactly the same from one trial to another.

An intuitive explanation of a binomial distribution is flipping a coin 10 times. If we have a fair coin our chance of getting heads (p) is 0.50. Now we throw the coin 10 times and count how many times it comes up heads. In most situations we will get heads 5 times, but there is also a change

that we get heads 9 times. The PMF of a binomial distribution will give these probabilities if we say $N = 10$ and $p = 0.5$. We say that the x for heads is 1 and 0 for tails.

PMF:

CDF:

A **Bernoulli Distribution** is a special case of a Binomial Distribution.

All values in a Bernoulli Distribution are either 0 or 1.

For example, if we take an unfair coin which falls on heads 60 % of the time, we can describe the Bernoulli distribution as follows:

p (change of heads) = 0.6

$1 - p$ (change of tails) = 0.4

heads = 1

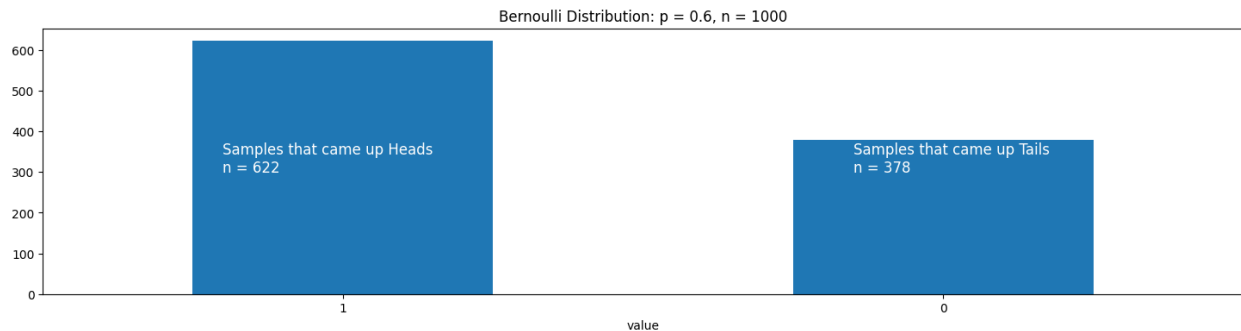
tails = 0

Formally, we can describe a Bernoulli distribution with the following PMF (Probability Mass Function):

```
# Change of heads (outcome 1)
p = 0.6

# Create Bernoulli samples
bern_dist = bernoulli.rvs(p, size=1000)
bern_df = pd.DataFrame({'value' : bern_dist})
bern_values = bern_df['value'].value_counts()

# Plot Distribution
plt.figure(figsize=(18,4))
bern_values.plot(kind='bar', rot=0)
plt.annotate(xy=(0.85,300),
             text='Samples that came up Tails\nn =',
             {}'.format(bern_values[0]),
             fontsize='large',
             color='white')
plt.annotate(xy=(-0.2,300),
             text='Samples that came up Heads\nn =',
             {}'.format(bern_values[1]),
             fontsize='large',
             color='white')
plt.title('Bernoulli Distribution: p = 0.6, n = 1000')
Text(0.5, 1.0, 'Bernoulli Distribution: p = 0.6, n = 1000')
```



```
bern_dist = bernoulli.rvs(p, size=1000)
```

Poisson Distribution

The Poisson distribution is a discrete distribution and is popular for modelling the number of times an event occurs in an interval of time or space.

It takes a value lambda, which is equal to the mean of the distribution.

PMF:

$$\frac{e^{-\lambda} \lambda^x}{x!}$$

e = Euler's constant ≈ 2.718
 λ = mean or expected value of the variable
 x = number of success for the event
 $!$ = factorial

$$\text{Poisson CDF} = \frac{\gamma(y, A)}{\Gamma(y)}$$

where: y = Test value, A = Expectation (mean)

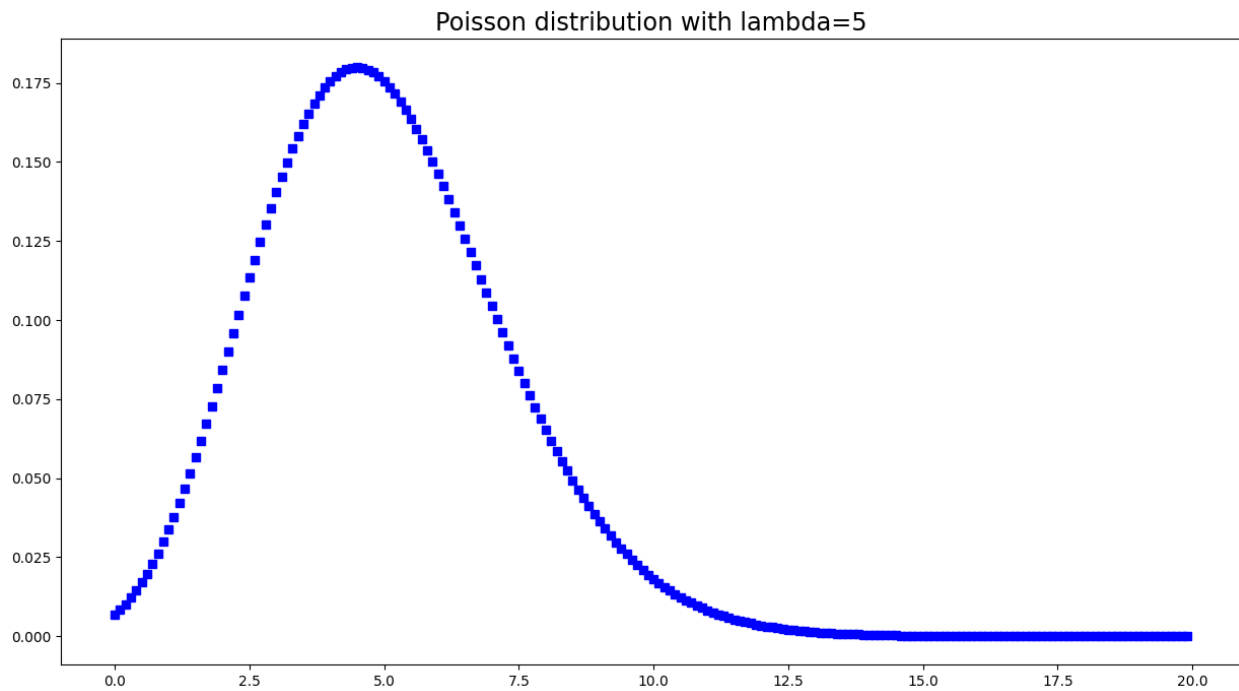
$$\text{and: } \Gamma(x) = \int_0^{\infty} t^{x-1} e^{-t} dt$$

$$\text{and: } \gamma(x, y) = \int_0^y t^{x-1} e^{-t} dt$$

CDF:

```
x = np.arange(0, 20, 0.1)
y = np.exp(-5)*np.power(5, x)/factorial(x)

plt.figure(figsize=(15,8))
plt.title('Poisson distribution with lambda=5', fontsize='xx-large')
plt.plot(x, y, 'bs')
plt.show()
```



Log-Normal Distribution

A log-normal distribution is continuous. The main characteristic of a log-normal distribution is that its logarithm is normally distributed. It is also referred to as Galton's distribution.

PDF:

$$f(x) = \frac{1}{x\sigma\sqrt{2\pi}} e^{\left(-\frac{(\ln(x)-\mu)^2}{2\sigma^2}\right)}$$

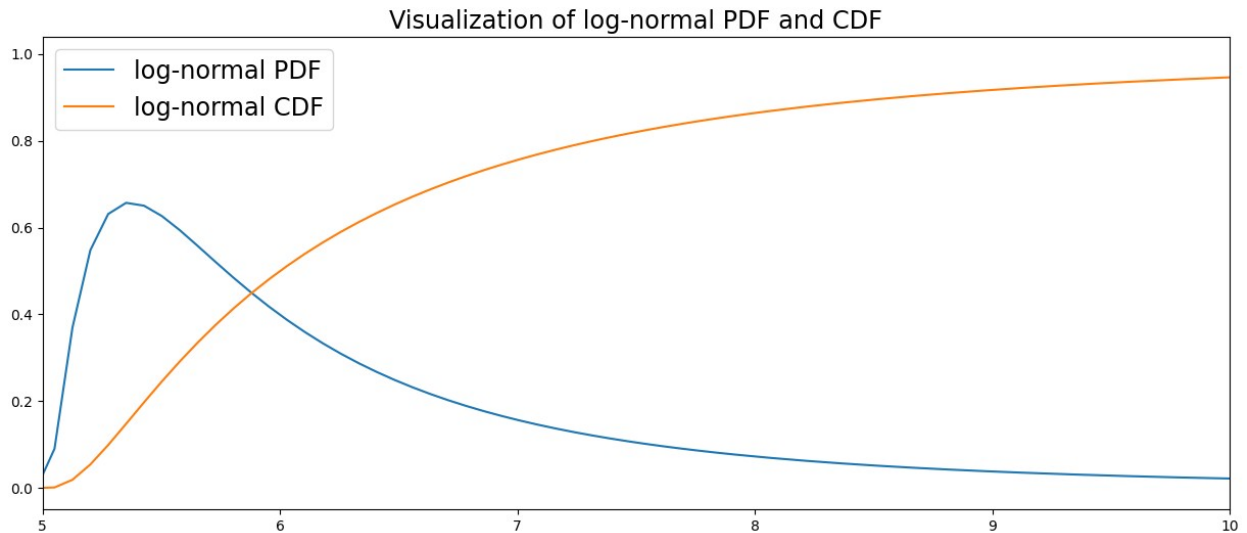
CDF:

Where Φ is the CDF of the standard normal distribution.

```
# Specify standard deviation and mean
std = 1
mean = 5

# Create log-normal distribution
dist=lognorm(std,loc=mean)
x=np.linspace(0,15,200)

# Visualize log-normal distribution
plt.figure(figsize=(15,6))
plt.xlim(5, 10)
plt.plot(x,dist.pdf(x), label='log-normal PDF')
plt.plot(x,dist.cdf(x), label='log-normal CDF')
plt.legend(fontsize='xx-large')
plt.title('Visualization of log-normal PDF and CDF', fontsize='xx-
large')
plt.show()
```



Summary Statistics and Moments

Mean, Median and Mode

Note: The mean is also called the first moment.

Central Tendency Measures		
Measure	Formula	Description
Mean	$\sum x/n$	Balance Point
Median	$n+1/2$ Position	Middle Value when ordered
Mode	None	Most frequent

Moments

A moment is a quantitative measure that says something about the shape of a distribution. There are central moments and non-central moments. This section is focused on the central moments.

The 0th central moment is the total probability and is always equal to 1.

The 1st moment is the mean (expected value).

The 2nd central moment is the variance.

Variance = The average of the squared distance of the mean. Variance is interesting in a mathematical sense, but the standard deviation is often a much better measure of how spread out the distribution is.

$$s^2 = \frac{\sum_{i=1}^n (Y_i - \bar{Y})^2}{(N - 1)}$$

Standard Deviation = The square root of the variance

$$s = \sqrt{\frac{\sum_{i=1}^n (Y_i - \bar{Y})^2}{(N - 1)}}$$

The 3rd central moment is the skewness.

Skewness = A measure that describes the contrast of one tail versus the other tail. For example, if there are more high values in your distribution than low values then your distribution is 'skewed' towards the high values.

$$skewness = \frac{\sum_{i=1}^n (Y_i - \bar{Y})^3}{(n - 1)s^3}$$

The 4th central moment is the kurtosis.

Kurtosis = A measure of how 'fat' the tails in the distribution are.

$$kurtosis = \frac{\sum_{i=1}^n (Y_i - \bar{Y})^4}{(n - 1)s^4}$$

The higher the moment, the harder it is to estimate with samples. Larger samples are required in order to obtain good estimates.

```
# Summary
print('Summary Statistics for a normal distribution: ')
# Median
medi = median(normal_dist)
print('Median: ', medi)
display(normal_df.describe())

# Standard Deviation
std = sqrt(np.var(normal_dist))

print('The first four calculated moments of a normal distribution: ')
# Mean
mean = normal_dist.mean()
print('Mean: ', mean)

# Variance
var = np.var(normal_dist)
print('Variance: ', var)

# Return unbiased skew normalized by N-1
skew = normal_df['value'].skew()
print('Skewness: ', skew)

# Return unbiased kurtosis over requested axis using Fisher's
definition of kurtosis
# (kurtosis of normal == 0.0) normalized by N-1
kurt = normal_df['value'].kurtosis()
print('Kurtosis: ', kurt)

Summary Statistics for a normal distribution:
Median:  0.007147454991141279

{"summary": "{\n  \"name\": \"print('Kurtosis: ', kurt)\",\n  \"rows\": 8,\n  \"fields\": [\n    {\n      \"column\": \"value\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 3535.467171654707,\n        \"min\": -3.4781782891317006,\n        \"max\": 10000.0,\n        \"num_unique_values\": 8,\n
```

```

{"samples": [{"x": -0.002594380156196375, "y": 0.007147454991141279, "z": 10000.0, "semantic_type": "description"}], "type": "dataframe"}

```

The first four calculated moments of a normal distribution:

```
Mean:    -0.002594380156196375
Variance: 0.9933382346280276
Skewness: -0.02493192024190024
Kurtosis: 0.0030777965889758896
```

Bias, MSE and SE

```
# Take sample
normal_df_sample = normal_df.sample(100)
```

```
# Calculate Expected Value (EV), population mean and bias
ev = normal_df_sample.mean()[0]
pop_mean = normal_df.mean()[0]
bias = ev - pop_mean
```

```
/tmp/ipykernel_9909/2703849781.py:5: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    ev = normal_df_sample.mean()[0]
/tmp/ipykernel_9909/2703849781.py:6: FutureWarning: Series.__getitem__
treating keys as positions is deprecated. In a future version, integer
keys will always be treated as labels (consistent with DataFrame
behavior). To access a value by position, use `ser.iloc[pos]`
    pop_mean = normal_df.mean()[0]
```

```
print('Sample mean (Expected Value): ', ev)
print('Population mean: ', pop_mean)
print('Bias: ', bias)
```

Sample mean (Expected Value): 0.24109677905387922
Population mean: -0.002594380156196375
Bias: 0.2436911592100756

MSE (Mean Squared Error) is a formula to measure how much estimators deviate from the true distribution. This can be very useful with for example, evaluating regression models.

RMSE (Root Mean Squared Error) is just the root of the MSE.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum (\hat{Y}_i - Y_i)^2}$$

```
from math import sqrt

Y = 100 # Actual Value
YH = 94 # Predicted Value

# MSE Formula
def MSE(Y, YH):
    return np.square(YH - Y).mean()

# RMSE formula
def RMSE(Y, YH):
    return sqrt(np.square(YH - Y).mean())

print('MSE: ', MSE(Y, YH))
print('RMSE: ', RMSE(Y, YH))

MSE: 36.0
RMSE: 6.0
```

The **Standard Error (SE)** measures how spread the distribution is from the sample mean.

The formula can also be defined as the standard deviation divided by the square root of the number of samples.

$$SE = \frac{\sigma}{\sqrt{n}}$$

← Standard deviation

← Number of samples

```
# Standard Error (SE)
uni_sample = uniform_dist.sample(100)
norm_sample = normal_dist.sample(100)

print('Standard Error of uniform sample: ', sem(uni_sample))
print('Standard Error of normal sample: ', sem(norm_sample))

# The random samples from the normal distribution should have a higher
standard error

Standard Error of uniform sample: 0.02770808414490087
Standard Error of normal sample: 0.11074612547748197
```

Sampling methods

Non-Representative Sampling:

Convenience Sampling = Pick samples that are most convenient, like the top of a shelf or people that can be easily approached.

Haphazard Sampling = Pick samples without thinking about it. This often gives the illusion take you are picking out samples at random.

Purposive Sampling = Pick samples for a specific purpose. An example is to focus on extreme cases. This can be useful but is limited because it doesn't allow you to make statements about the whole population.

Representative Sampling:

Simple Random Sampling = Pick samples (psuedo)randomly.

Systematic Sampling = Pick samples with a fixed interval. For example every 10th sample (0, 10, 20, etc.).

Stratified Sampling = Pick the same amount of samples from different groups (strata) in the population.

Cluster Sampling = Divide the population into groups (clusters) and pick samples from those groups.

```
# Note that we take very small samples just to illustrate the different sampling methods

print('---Non-Representative samples:---\n')
# Convenience samples
con_samples = normal_dist[0:5]
print('Convenience samples:\n\n{}\n'.format(con_samples))

# Haphazard samples (Picking out some numbers)
hap_samples = [normal_dist[12], normal_dist[55], normal_dist[582],
normal_dist[821], normal_dist[999]]
print('Haphazard samples:\n\n{}\n'.format(hap_samples))

# Purposive samples (Pick samples for a specific purpose)
# In this example we pick the 5 highest values in our distribution
purp_samples = normal_dist.nlargest(n=5)
print('Purposive samples:\n\n{}\n'.format(purp_samples))

print('---Representative samples:---\n')

# Simple (pseudo)random sample
rand_samples = normal_dist.sample(5)
print('Random samples:\n\n{}\n'.format(rand_samples))

# Systematic sample (Every 2000th value)
sys_samples = normal_dist[normal_dist.index % 2000 == 0]
print('Systematic samples:\n\n{}\n'.format(sys_samples))

# Stratified Sampling
# We will get 1 person from every 'housing_median_age' group in the dataset
# Using 'median_income' as the sampled value
```

```
df = pd.read_csv('/content/sample_data/california_housing_train.csv')
strat_samples = []

for age_group in df['housing_median_age'].unique():
    samp = df[df['housing_median_age'] == age_group].sample(1)
    strat_samples.append(samp['median_income'].item())

print('Stratified samples:\n\n{}\n'.format(strat_samples))
```

```
# Cluster Sampling
# Make random clusters of ten people (Here with replacement)
c1 = normal_dist.sample(10)
c2 = normal_dist.sample(10)
c3 = normal_dist.sample(10)
c4 = normal_dist.sample(10)
c5 = normal_dist.sample(10)

# Take sample from every cluster (with replacement)
clusters = [c1,c2,c3,c4,c5]
cluster_samples = []
for c in clusters:
    clus_samp = c.sample(1)
    cluster_samples.extend(clus_samp)
print('Cluster samples:\n\n{}\n'.format(cluster_samples))
```

---Non-Representative samples:---

Convenience samples:

```
0    -0.729045
1     0.906140
2     1.231654
3     1.162371
4     1.125285
dtype: float64
```

Haphazard samples:

```
[np.float64(-0.6049544892094189), np.float64(0.3173800241723485),
np.float64(1.710145196050765), np.float64(-1.2211122259530578),
np.float64(0.40274869147587394)]
```

Purposive samples:

```
5970    3.801432
1404    3.709553
1263    3.652398
2164    3.555515
2834    3.358059
dtype: float64
```

```
---Representative samples:---
```

```
Random samples:
```

```
1977    -0.223335
978      1.468293
3621     0.625964
4642    -0.349270
5678     0.333560
dtype: float64
```

```
Systematic samples:
```

```
0        -0.729045
2000      0.830696
4000     -1.084550
6000      1.019317
8000      0.387380
dtype: float64
```

```
Stratified samples:
```

```
[2.2278, 2.1006, 1.6214, 3.8244, 2.575, 12.3292, 4.1483, 3.1779,
4.3194, 5.5196, 6.7064, 1.9923, 0.9948, 3.8875, 4.375, 5.6344, 2.3713,
5.8521, 8.5759, 3.2303, 3.0, 3.285, 5.3146, 4.6957, 5.0762, 4.2788,
1.4817, 4.7147, 3.0156, 4.4453, 3.2396, 3.5714, 2.2692, 2.1187,
4.4722, 3.9443, 2.8042, 3.6033, 4.9196, 4.95, 5.4284, 5.7245, 2.8015,
2.8667, 1.3479, 5.7248, 4.0, 2.6364, 1.7945, 3.8047, 9.3694, 4.25]
```

```
Cluster samples:
```

```
[-1.817664969466525, 1.1398718156926395, -0.01188981657942072,
0.008699275091198085, -0.263212250364347]
```

Covariance

Covariance is a measure of how much two random variables vary together. variance is similar to covariance in that variance shows you how much one variable varies. Covariance tells you how two variables vary together.

If two variables are independent, their covariance is 0. However, a covariance of 0 does not imply that the variables are independent.

```
# Covariance between Age and Income
print('Covariance between Housing Median Age and Median Income: ')

df[['housing_median_age', 'median_income']].cov()

Covariance between Housing Median Age and Median Income:
```

```
{
  "summary": {
    "name": "df[['housing_median_age', 'median_income']]",
    "rows": 2,
    "fields": [
      {
        "column": "housing_median_age",
        "properties": {
          "dtype": "number",
          "std": 113.99650997455741,
          "min": -2.784427888928761,
          "max": 158.43098258029013,
          "num_unique_values": 2,
          "samples": [
            2.784427888928761,
            158.43098258029013
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "median_income",
        "properties": {
          "dtype": "number",
          "std": 4.5435069769655225,
          "min": -2.784427888928761,
          "max": 3.641061298632662,
          "num_unique_values": 2,
          "samples": [
            2.784427888928761,
            3.641061298632662
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ],
    "type": "dataframe"
  }
}
```

Correlation

Correlation is a standardized version of covariance. Here it becomes more clear that Age and Income do not have a strong correlation in our dataset.

The formula for Pearson's correlation coefficient consists of the covariance between the two random variables divided by the standard deviation of the first random variable times the standard deviation of the second random variable.

Formula for Pearson's correlation coefficient:

```
# Correlation between two normal distributions
# Using Pearson's correlation
print('Pearson: ')
print(df[['housing_median_age', 'median_income']].corr(method='pearson'))
```

```

# Negatively correlated data
x1 = [1, 3, 3, 4, 6, 7, 7, 9, 10, 10]
x2 = [9.7, 9.3, 7, 6, 7, 7.2, 6, 4.2, 4.5, 4]

# Positively correlated data
y1 = [0, 1, 2, 2.3, 3, 4, 4, 7, 9, 9.5]
y2 = [2.2, 4, 3, 5, 5.3, 5, 7, 9, 5, 10]

# Correlation with NumPy
print('\nx1,x2 correlation: \n', np.corrcoef(x1, x2))
print('\ny1,y2 correlation: \n', np.corrcoef(y1, y2))

Pearson:

```

	housing_median_age	median_income
housing_median_age	1.000000	-0.115932
median_income	-0.115932	1.000000

```

x1,x2 correlation:
[[ 1.          -0.88237093]
 [-0.88237093  1.          ]]

y1,y2 correlation:
[[1.          0.7737071]
 [0.7737071  1.          ]]

# Using Spearman's rho correlation
print('Spearman: ')
print(df[['housing_median_age',
'median_income']].corr(method='spearman'))

Spearman:

```

	housing_median_age	median_income
housing_median_age	1.000000	-0.143134
median_income	-0.143134	1.000000

Linear regression

Linear Regression can be performed through Ordinary Least Squares (OLS) or Maximum Likelihood Estimation (MLE).

Most Python libraries use OLS to fit linear models.

Simple Linear Regression Model

The diagram illustrates the Simple Linear Regression Model equation: $Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$. Each term is labeled with an arrow pointing to it: Y_i is the Dependent Variable; β_0 is the Population Y intercept; β_1 is the Population Slope Coefficient; X_i is the Independent Variable; and ϵ_i is the Random Error term. Below the equation, two blue curly braces group the terms: the first brace under $\beta_0 + \beta_1 X_i$ is labeled 'Linear component', and the second brace under ϵ_i is labeled 'Random Error component'.

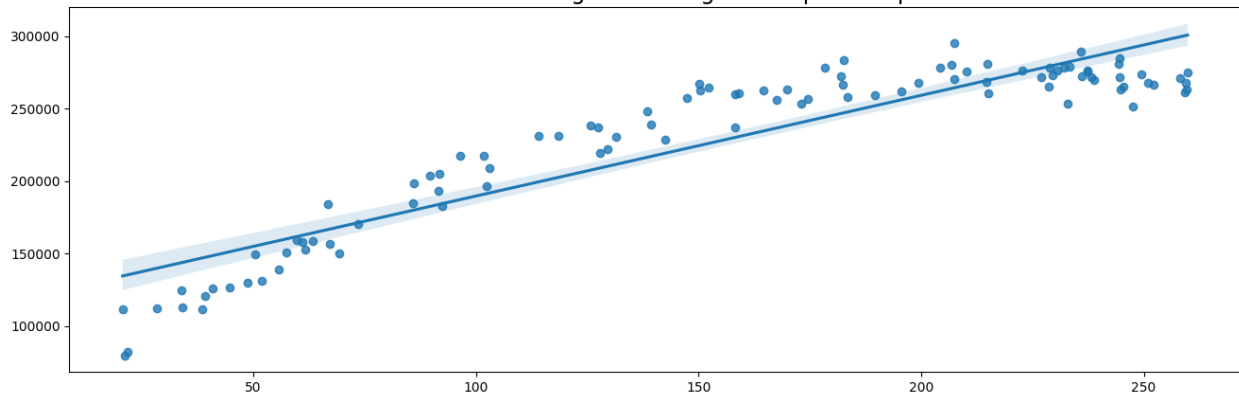


```
# Generate data
x = np.random.uniform(low=20, high=260, size=100)
y = 50000 + 2000*x - 4.5 * x**2 + np.random.normal(size=100, loc=0,
scale=10000)

# Plot data with Linear Regression
plt.figure(figsize=(16,5))
plt.title('Well fitted but not well fitting: Linear regression plot on
quadratic data', fontsize='xx-large')
sns.regplot(x=x, y=y)

<Axes: title={'center': 'Well fitted but not well fitting: Linear
regression plot on quadratic data'}>
```

Well fitted but not well fitting: Linear regression plot on quadratic data



```
# Linear regression from scratch
import random
# Create data from regression
xs = np.array(range(1,20))
ys = [0,8,10,8,15,20,26,29,38,35,40,60,50,61,70,75,80,88,96]

# Put data in dictionary
data = dict()
for i in list(xs):
    data.update({xs[i-1] : ys[i-1]})

# Slope
m = 0
# y intercept
b = 0
# Learning rate
lr = 0.0001
# Number of epochs
epochs = 100000

# Formula for linear line
def lin(x):
    return m * x + b

# Linear regression algorithm
for i in range(epochs):
    # Pick a random point and calculate vertical distance and
    horizontal distance
    rand_point = random.choice(list(data.items()))
    vert_dist = abs((m * rand_point[0] + b) - rand_point[1])
    hor_dist = rand_point[0]

    if (m * rand_point[0] + b) - rand_point[1] < 0:
        # Adjust line upwards
        m += lr * vert_dist * hor_dist
        b += lr * vert_dist
```

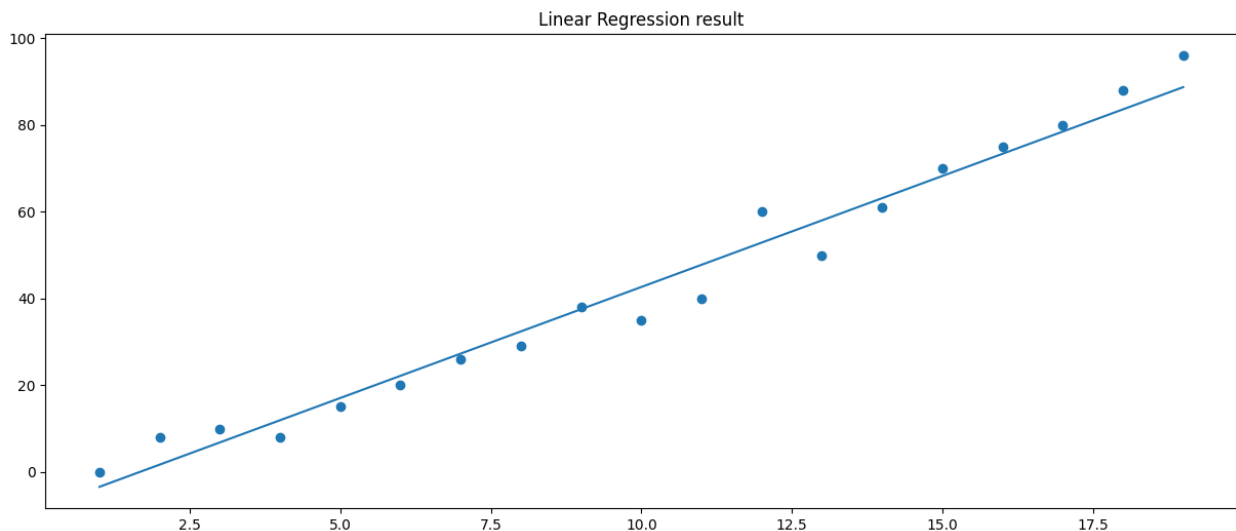
```

else:
    # Adjust line downwards
    m -= lr * vert_dist * hor_dist
    b -= lr * vert_dist

# Plot data points and regression line
plt.figure(figsize=(15,6))
plt.scatter(data.keys(), data.values())
plt.plot(xs, lin(xs))
plt.title('Linear Regression result')
print('Slope: {}\nIntercept: {}'.format(m, b))

```

Slope: 5.119713622194183
Intercept: -8.530281187819778



Anscombe's Quartet

Anscombe's quartet is a set of four datasets that have the same descriptive statistics and linear regression fit. The datasets are however very different from each other.

This sketches the issue that although summary statistics and regression models are really helpful in understanding your data, you should always visualize the data to see what's really going on. It also shows that a few outliers can really mess up your model.

[More information on Anscombe's Quartet](#)

Bootstrapping

Bootstrapping is a resampling technique to quantify the uncertainty of an estimator given sample data. In other words, we have a sample of data and we take multiple samples from that sample. For example, with bootstrapping we can take means for each bootstrap sample and thereby make a distribution of means.

Once we created a distribution of estimators we can use this to make decisions.

Bootstrapping can be:

1. Non-parametric (Take random samples from sample)
2. Parametric (Use the sample to create new datasets with a (normal) distribution which has sample mean and variance). Downside: You are making assumptions about the distribution. Upside: Computationally more light
3. Online bootstrap (Take samples from a stream of data)

The following code implements a simple non-parametric bootstrap to create a distribution of means, medians and midranges of the Income distribution in our Toy Dataset. We can use this to deduce which income means will make sense for subsequent samples.

```
# scikit-learn bootstrap package
from sklearn.utils import resample

# data sample
data = df['median_income'].sample(10000)

# prepare bootstrap samples
boot = resample(data, replace=True, n_samples=10, random_state=1)
print('Means of Bootstrap Samples: \n{}\n'.format(boot))
print('Mean of the population: ', data.mean())
print('Standard Deviation of the population: ', data.std())
```

```
# Bootstrap plot
pd.plotting.bootstrap_plot(data)
```

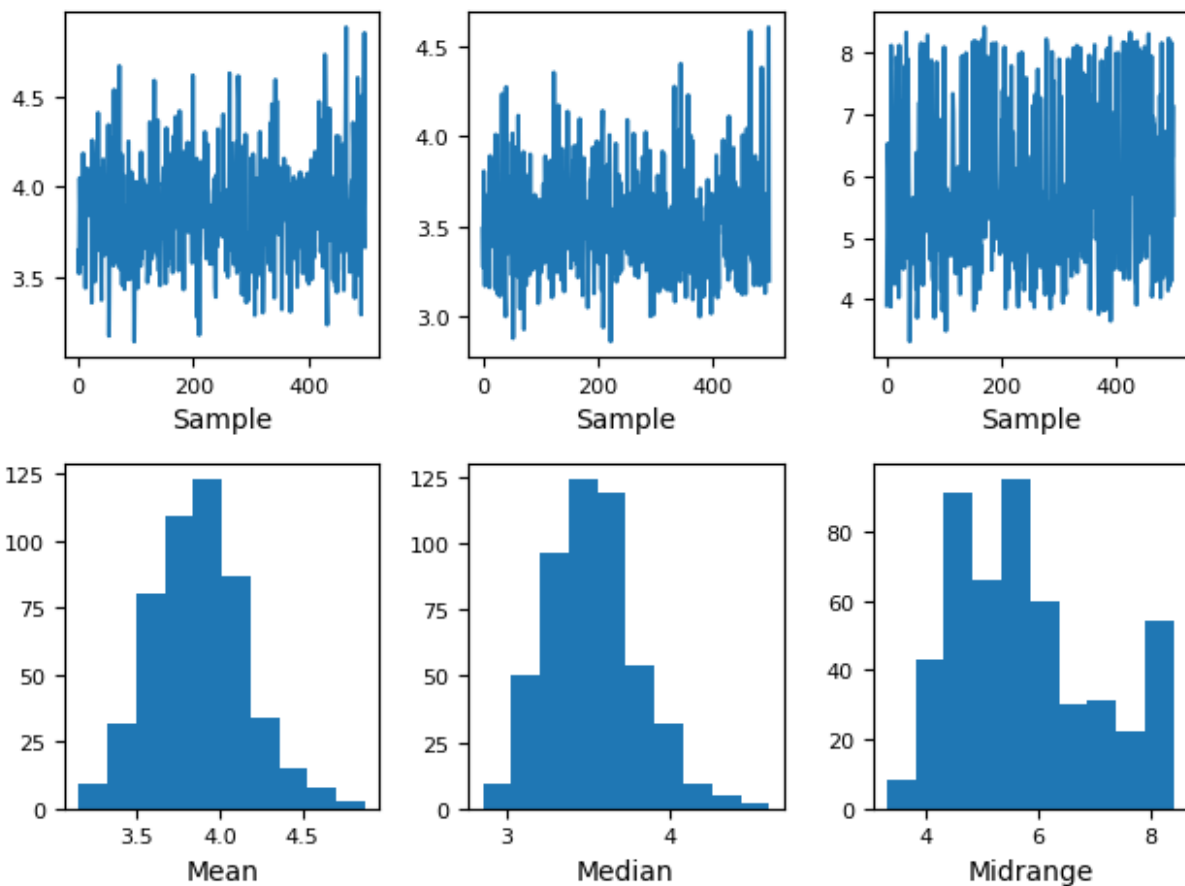
Means of Bootstrap Samples:

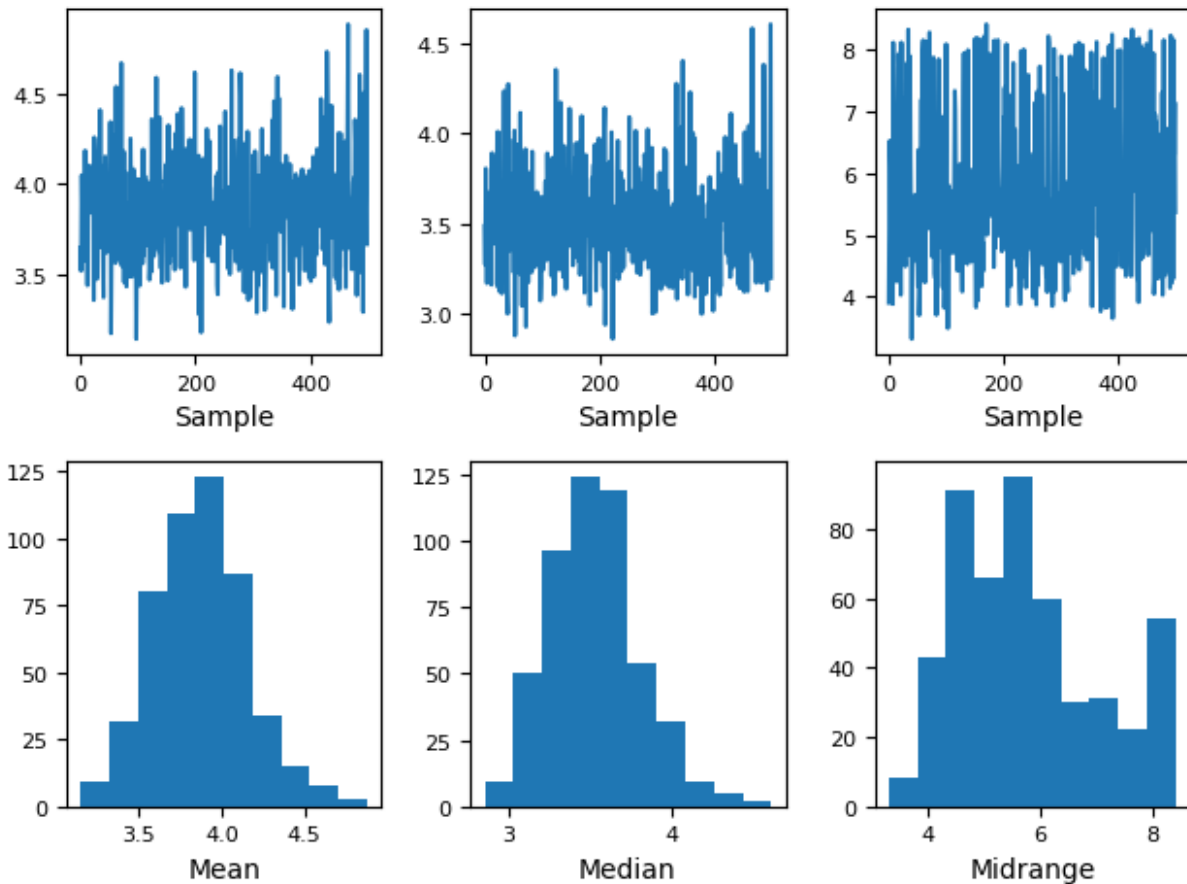
9877	2.6250
8640	4.6731
2587	5.4188
14922	2.3833
5457	1.8715
10527	2.3506
9251	1.5875
2530	1.7292
14303	10.0259
5966	1.7778

Name: median_income, dtype: float64

Mean of the population: 3.87378067

Standard Deviation of the population: 1.8965652483153508





Hypothesis testing

We establish two hypotheses, H_0 (Null hypothesis) and H_a (Alternative Hypothesis).

We can make four different decisions with hypothesis testing:

1. Reject H_0 and H_0 is not true (no error)
2. Do not reject H_0 and H_0 is true (no error)
3. Reject H_0 and H_0 is true (Type 1 Error)
4. Do not reject H_0 and H_0 is not true (Type 2 error)

Type 1 error is also called Alpha error. Type 2 error is also called Beta error.

	<i>Decision</i>	
	Accept H_0	Reject H_0
H_0 (true)	Correct decision	Type I error (α error)
H_0 (false)	Type II error (β error)	Correct decision

P-Value

A p-value is the probability of finding equal or more extreme results when the null hypothesis (H_0) is true. In other words, a low p-value means that we have compelling evidence to reject the null hypothesis.

If the p-value is lower than 5% ($p < 0.05$). We often reject H_0 and accept H_a is true. We say that $p < 0.05$ is statistically significant, because there is less than 5% chance that we are wrong in rejecting the null hypothesis.

One way to calculate the p-value is through a T-test. We can use [Scipy's ttest_ind function](#) to calculate the t-test for the means of two independent samples of scores. In this example we calculate the t-statistic and p-value of two random samples 10 times.

We see that the p-value is sometimes very low, but this does not mean that these two random samples are correlated. This is why you have to be careful with relying too heavily of p-values. If you repeat an experiment multiple times you can get trapped in the illusion that there is correlation where there is only randomness.

This xkcd comic perfectly illustrates the hazards of relying too much on p-values.

```
# Perform t-test and compute p value of two random samples
print('T-statistics and p-values of two random samples.')
for _ in range(10):
    rand_sample1 = np.random.random_sample(10)
    rand_sample2 = np.random.random_sample(10)
    print(stats.ttest_ind(rand_sample1, rand_sample2))

T-statistics and p-values of two random samples.
TtestResult(statistic=np.float64(-0.8251685068726443),
pvalue=np.float64(0.4200778608037121), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.26154035568078077),
pvalue=np.float64(0.79664507561244), df=np.float64(18.0))
TtestResult(statistic=np.float64(-2.220370342307992),
pvalue=np.float64(0.03947063563883202), df=np.float64(18.0))
TtestResult(statistic=np.float64(1.0841710986761517),
pvalue=np.float64(0.2925977243924466), df=np.float64(18.0))
TtestResult(statistic=np.float64(-1.1662928847653413),
pvalue=np.float64(0.2587169831229167), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.9459277604717623),
pvalue=np.float64(0.35671164084353635), df=np.float64(18.0))
TtestResult(statistic=np.float64(-0.9071232708371005),
pvalue=np.float64(0.3763259945151387), df=np.float64(18.0))
TtestResult(statistic=np.float64(-1.2170740849762403),
pvalue=np.float64(0.23929307658485038), df=np.float64(18.0))
TtestResult(statistic=np.float64(1.0850895583660198),
pvalue=np.float64(0.2922016580022174), df=np.float64(18.0))
TtestResult(statistic=np.float64(0.6810263681354581),
pvalue=np.float64(0.5045224626728317), df=np.float64(18.0))
```

q-q plot (quantile-quantile plot)

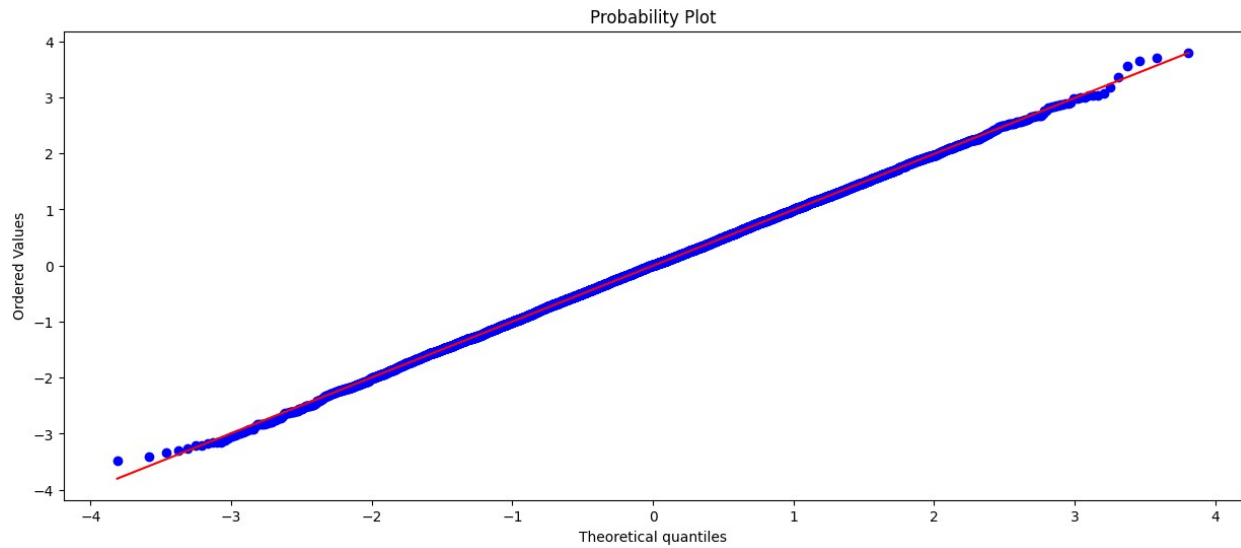
Many statistical techniques require that data is coming from a normal distribution (for example, t-test). Therefore, it is important to verify this before applying statistical techniques.

One approach is to visualize and make a judgment about the distribution. A q-q plot is very helpful for determining if a distribution is normal. There are other tests for testing 'normality', but this is beyond the scope of this tutorial.

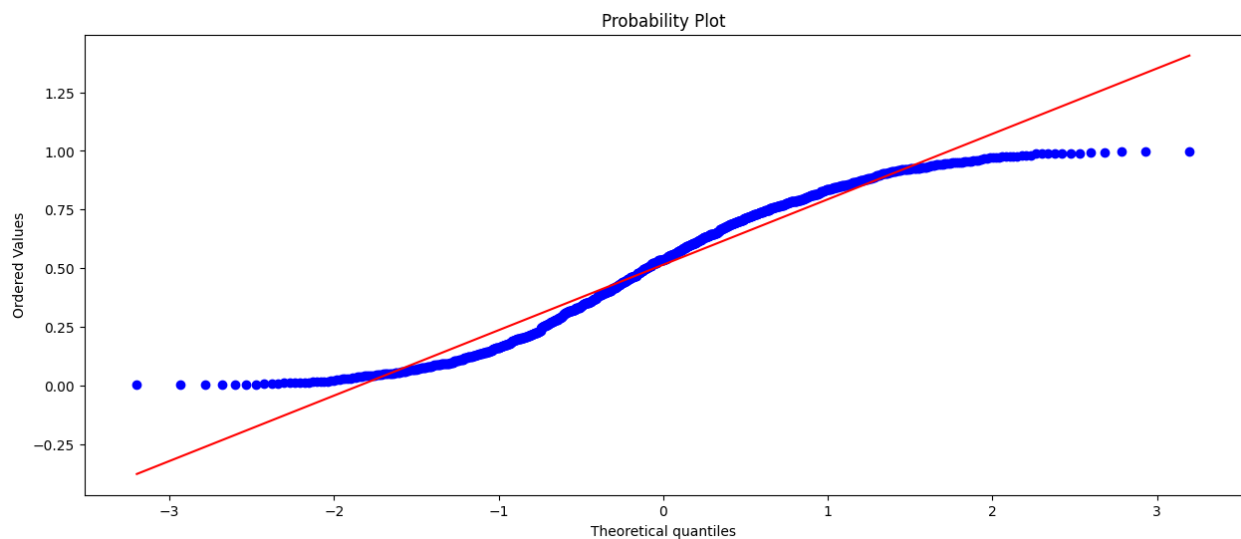
In the first plot we can easily see that the values line up nicely. From this we conclude that the data is normally distributed.

In the second plot we can see that the values don't line up. Our conclusion is that the data is not normally distributed. In this case the data was uniformly distributed.

```
# q-q plot of a normal distribution
plt.figure(figsize=(15,6))
stats.probplot(normal_dist, dist="norm", plot=plt)
plt.show()
```



```
# q-q plot of a uniform/random distribution
plt.figure(figsize=(15,6))
stats.probplot(uniform_dist, dist="norm", plot=plt)
plt.show()
```



Outliers

In Grubbs test, the null hypothesis is that no observation is an outlier, while the alternative hypothesis is that there is one observation an outlier. Thus the Grubbs test is only searching for one outlier observation.

The formula for Grubbs test:

Where $\bar{Y}$ is the sample mean and s is the standard deviation. The Grubbs test statistic is the largest absolute deviation from the sample mean in units of the sample standard deviation.

[Source](#)

Tukey's method

Tukey suggested that an observation is an outlier whenever an observation is 1.5 times the interquartile range below the first quartile or 1.5 times the interquartile range above the third quartile. This may sound complicated, but is quite intuitive if you see it visually.

For normal distributions, Tukey's criteria for outlier observations is unlikely if no outliers are present, but using Tukey's criteria for other distributions should be taken with a grain of salt.

The formula for Tukey's method:

Y_a is the larger of two means being compared. SE is the standard error of the sum of the means.

[Source](#)

```

# Detect outliers on the 'Income' column of the Toy Dataset

# Function for detecting outliers a la Tukey's method using z-scores
def tukey_outliers(data) -> list:
    # For more information on calculating the threshold check out:
    # https://medium.com/datadriveninvestor/finding-outliers-in-dataset-using-python-efc3fce6ce32
    threshold = 3

    mean = np.mean(data)
    std = np.std(data)

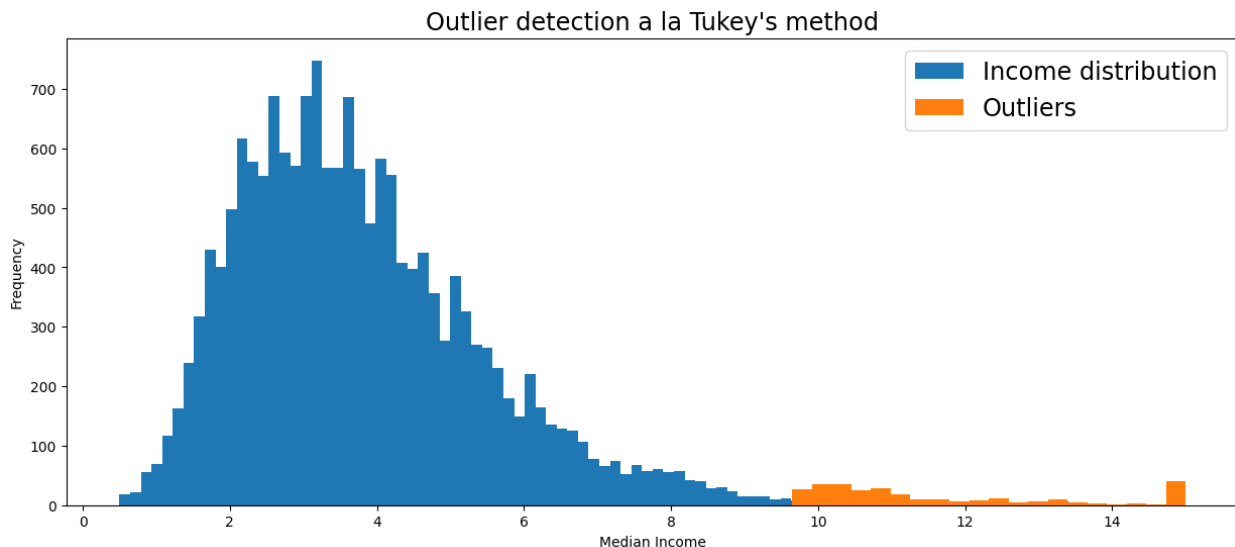
    # Spot and collect outliers
    outliers = []
    for i in data:
        z_score = (i - mean) / std
        if abs(z_score) > threshold:
            outliers.append(i)
    return outliers

# Get outliers
income_outliers = tukey_outliers(df['median_income'])

# Visualize distribution and outliers
plt.figure(figsize=(15,6))
df['median_income'].plot(kind='hist', bins=100, label='Income distribution')
plt.hist(income_outliers, bins=20, label='Outliers')
plt.title("Outlier detection a la Tukey's method", fontsize='xx-large')
plt.xlabel('Median Income')
plt.legend(fontsize='xx-large')

<matplotlib.legend.Legend at 0x7e72c853d0d0>

```



Overfitting

Our model is overfitting if it is also modeling the 'noise' in the data. This implies that the model will not generalize well to new data even though the error on the training data becomes very small. Linear models are unlikely to overfit, but as models become more flexible we have to be wary of overfitting. Our model can also underfit which means that it has a large error on the training data.

Finding the sweet spot between overfitting and underfitting is called the [Bias Variance Trade-off](#). It is nice to know this theorem, but more important to understand how to prevent it. I will explain some techniques for how to do this below.

Prevention of Overfitting

1. Split data into training data and test data.
2. Regularization: limit the flexibility of the model.
3. Cross Validation

Cross Validation

Cross validation is a technique to estimate the accuracy of our statistical model. It is also called out-of-sample testing or rotation estimation. Cross validation will help us to recognize overfitting and to check if our model generalizes to new (out-of-sample) data.

A popular cross validation technique is called k-fold cross validation. The idea is simple, we split our dataset up in k datasets and out of each dataset k we pick out a few samples. We then fit our model on the rest of k and try to predict the samples we picked out. We use a metric like Mean Squared Error to estimate how good our predictions are. This procedure is repeated and then we look at the average of the predictions over multiple cross-validation data sets.

A special case where we pick out one samples is called 'Leave-One-Out Cross Validation (LOOCV)'. However, the variance of LOOCV is high.

For more information about cross validation [check out this blog](#).

Generalized Linear Models (GLMs)

Link functions

A Link Function is used in Generalized Linear Models (GLMs) to apply linear models for a continuous response variable given continuous and/or categorical predictors. A link function that is often used is called the inverse logit or logistic sigmoid function.

The link function provides a relationship between the linear predictor and the mean of a distribution.

Logistic regression

With logistic regression we use a link function like the inverse logit function mentioned above to model a binary dependent variable. While a linear regression model predicts the expected value of y given x directly, a GLM uses a link function.

We can easily implement logistic regression with [sklearn's Logistic Regression function](#).

```
# Simple example of Logistic Regression in Python
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
X, y = load_iris(return_X_y=True)

# Logistic regression classifier
clf = LogisticRegression(random_state=0,
                        solver='lbfgs',
                        multi_class='multinomial').fit(X, y)

print('Accuracy score of logistic regression model on the Iris flower
dataset: {}'.format(clf.score(X, y)))
```

```
Accuracy score of logistic regression model on the Iris flower
dataset: 0.9733333333333334
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated in
version 1.5 and will be removed in 1.7. From then on, it will always
use 'multinomial'. Leave it to its default value to avoid this
warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic
.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (`max_iter`) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:

```
https://scikit-learn.org/stable/modules/linear_model.html#logistic-  
regression  
n_iter_i = _check_optimize_result(
```

Frequentist vs. Bayes

Frequentist:

1. Fixed parameters (Processes are fixed)
2. Repeated sampling leads to Probabilities

Bayes:

1. Probability as "degree of belief"
2. P(parameter) leads to all plausible values of the parameter
3. Updates degree of belief based on a prior belief

Frequentists and Bayesians agree that Bayes' Theorem is valid. See figure below for explanation of this theorem.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

Bayes theorem extends to distributions and random variables.